

TUGAS PAKTIKUM 4 – CPU Scheduling
SISTEM OPERASI



Dosen Pengampu:
Triawan Adi Cahyanto M.Kom

Disusun Oleh:
Nabila Senja Ramadhani (2410651043)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas 1: Eksperimen dengan Data Berbeda

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

P1: AT=0, BT=10

P2: AT=1, BT=5

P3: AT=2, BT=8

Data Set 2:

P1: AT=0, BT=5

P2: AT=1, BT=3

P3: AT=2, BT=8

P4: AT=3, BT=6

```

senjaramadhani@LAPTOP-823QCJCF:~$ nano cpu_scheduling.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc cpu_scheduling.c -o cpu_scheduling
senjaramadhani@LAPTOP-823QCJCF:~$ ./cpu_scheduling
DATA SET 1:

```

```

=== FCFS ===
Proses  AT      BT      WT      TAT      CT
P1       0      10      0      10      10
P2       1       5       9      14      15
P3       2       8      13      21      23

```

```

=== SJF (Non-preemptive) ===
Proses  AT      BT      WT      TAT      CT
P1       0      10      0      10      10
P2       1       5       9      14      15
P3       2       8      13      21      23

```

```

=== Round Robin (q=4) ===
Proses  AT      BT      WT      TAT      CT
P1       0      10      15      25      25
P2       1       5      12      17      18
P3       2       8      12      20      22

```

DATA SET 2:

```

=== FCFS ===
Proses  AT      BT      WT      TAT      CT
P1       0       5       0       5       5
P2       1       3       4       7       8
P3       2       8       6      14      16
P4       3       6      13      19      22

```

```

=== SJF (Non-preemptive) ===
Proses  AT      BT      WT      TAT      CT
P1       0       5       0       5       5
P2       1       3       4       7       8
P3       2       8      12      20      22
P4       3       6       5      11      14

```

DATA SET 2:

```

=== FCFS ===
Proses  AT      BT      WT      TAT      CT
P1       0       5       0       5       5
P2       1       3       4       7       8
P3       2       8       6      14      16
P4       3       6      13      19      22

```

```

=== SJF (Non-preemptive) ===
Proses  AT      BT      WT      TAT      CT
P1       0       5       0       5       5
P2       1       3       4       7       8
P3       2       8      12      20      22
P4       3       6       5      11      14

```

```

=== Round Robin (q=4) ===
Proses  AT      BT      WT      TAT      CT
P1       0       5      12      17      17
P2       1       3       3       6       7
P3       2       8      11      19      21
P4       3       6      14      20      23
senjaramadhani@LAPTOP-823QCJCF:~$ |

```

Source code:

```
#include <stdio.h>
```

```

struct Process {
    char name[3];
    int arrival, burst, waiting, turnaround, completion;
};

```

```
// Fungsi FCFS
```

```
void fcfs(struct Process p[], int n) {
```

```

int time = 0;
for(int i=0;i<n;i++){
    if(time < p[i].arrival)
        time = p[i].arrival;
    p[i].waiting = time - p[i].arrival;
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = p[i].completion - p[i].arrival;
}

printf("\n=== FCFS ===\n");
printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");
for(int i=0;i<n;i++){
    printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrival, p[i].burst, p[i].waiting,
p[i].turnaround, p[i].completion);
}
}

```

// Fungsi SJF Non-preemptive

```

void sjf(struct Process p[], int n) {
    struct Process temp;
    int completed = 0, time = 0, min_bt, pos;
    int isCompleted[n];
    for(int i=0;i<n;i++) isCompleted[i] = 0;

    printf("\n=== SJF (Non-preemptive) ===\n");
    while(completed != n) {
        min_bt = 9999;
        pos = -1;
        for(int i=0;i<n;i++) {
            if(p[i].arrival <= time && isCompleted[i]==0 && p[i].burst < min_bt) {

```

```

        min_bt = p[i].burst;
        pos = i;
    }
}
if(pos == -1) {
    time++;
    continue;
}
p[pos].waiting = time - p[pos].arrival;
time += p[pos].burst;
p[pos].completion = time;
p[pos].turnaround = p[pos].completion - p[pos].arrival;
isCompleted[pos] = 1;
completed++;
}

printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");
for(int i=0;i<n;i++){
    printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrival, p[i].burst, p[i].waiting,
p[i].turnaround, p[i].completion);
}
}

```

// Fungsi Round Robin

```

void rr(struct Process p[], int n, int quantum) {
    int remaining[n];
    int time = 0, done;
    for(int i=0;i<n;i++) remaining[i] = p[i].burst;

    printf("\n=== Round Robin (q=%d) ===\n", quantum);
    while(1){

```

```

done = 1;
for(int i=0;i<n;i++){
    if(remaining[i] > 0 && p[i].arrival <= time){
        done = 0;
        if(remaining[i] > quantum){
            time += quantum;
            remaining[i] -= quantum;
        } else {
            time += remaining[i];
            p[i].completion = time;
            p[i].turnaround = p[i].completion - p[i].arrival;
            p[i].waiting = p[i].turnaround - p[i].burst;
            remaining[i] = 0;
        }
    }
}
if(done == 1)
    break;
time++;
}

printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");
for(int i=0;i<n;i++){
    printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrival, p[i].burst, p[i].waiting,
p[i].turnaround, p[i].completion);
}
}

int main() {
    // DATA SET 1
    struct Process data1[] = {"P1",0,10}, {"P2",1,5}, {"P3",2,8}};

```

```

int n1 = 3;

printf("DATA SET 1:\n");
fcfs(data1, n1);
sjf(data1, n1);
rr(data1, n1, 4);

// DATA SET 2
struct Process data2[] = {{"P1",0,5}, {"P2",1,3}, {"P3",2,8}, {"P4",3,6}};
int n2 = 4;

printf("\n\nDATA SET 2:\n");
fcfs(data2, n2);
sjf(data2, n2);
rr(data2, n2, 4);

return 0;
}

```

Tugas 2: Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

Biasanya SJF (Shortest Job First) memiliki AWT terkecil karena proses dengan burst time paling pendek dijalankan lebih dulu.

2. Mengapa hasil algoritma berbeda-beda?

Karena setiap algoritma memiliki strategi penjadwalan berbeda:

- FCFS: Berdasarkan urutan kedatangan
- SJF: Berdasarkan lama waktu eksekusi (burst time)
- RR: Bergiliran dengan kuantum waktu tetap

3. Kapan sebaiknya menggunakan algoritma FCFS?

Ketika semua proses memiliki burst time hampir sama

4. Kapan sebaiknya menggunakan algoritma SJF?

Ketika diketahui atau dapat diperkirakan burst time tiap proses

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Ketika semua proses harus mendapat kesempatan eksekusi yang adil

Tugas 3: Modifikasi Program

1. Tambahkan fitur berikut pada salah satu program:
2. Tampilkan Gantt Chart yang lebih detail
3. Simpan hasil ke file text
4. Baca input dari file

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano cpu_scheduling_mod.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc cpu_scheduling_mod.c -o cpu_scheduling_mod
senjaramadhani@LAPTOP-823QCJCF:~$ nano input.txt
senjaramadhani@LAPTOP-823QCJCF:~$ ./cpu_scheduling_mod
Data berhasil dibaca dari file input.txt

=== FCFS ===
Proses  AT    BT    WT    TAT    CT
Gantt Chart:
| P1 | P2 | P3 |
0      10    15    23

Proses  AT    BT    WT    TAT    CT
P1      0     10     0     10     10
P2      1     5     9     14     15
P3      2     8     13     21     23
Average WT = 7.33
Average TAT = 15.00

Hasil juga disimpan ke file output.txt
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Process {
    char name[5];
    int arrival, burst, waiting, turnaround, completion;
};
```

// ----- FCFS dengan Gantt Chart + File Output -----

```
void fcfs(struct Process p[], int n, FILE *fout) {
    int time = 0;
    float totalWT = 0, totalTAT = 0;

    fprintf(fout, "=== FCFS ===\n");
    fprintf(fout, "Proses\tAT\tBT\tWT\tTAT\tCT\n");

    printf("\n=== FCFS ===\n");
    printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");
```

// Gantt Chart

```

printf("\nGantt Chart:\n");
fprintf(fout, "\nGantt Chart:\n");

for(int i=0; i<n; i++) {
    if(time < p[i].arrival)
        time = p[i].arrival;

    p[i].waiting = time - p[i].arrival;
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = p[i].completion - p[i].arrival;

    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;

    printf(" %s |", p[i].name);
    fprintf(fout, " %s |", p[i].name);
}

printf("\n0");
fprintf(fout, "\n0");

time = 0;
for(int i=0; i<n; i++) {
    if(time < p[i].arrival)
        time = p[i].arrival;
    time += p[i].burst;
    printf("\t%d", time);
    fprintf(fout, "\t%d", time);
}

printf("\n\nProses\tAT\tBT\tWT\tTAT\tCT\n");
for(int i=0; i<n; i++) {
    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].name, p[i].arrival, p[i].burst,
        p[i].waiting, p[i].turnaround, p[i].completion);

    fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].name, p[i].arrival, p[i].burst,
        p[i].waiting, p[i].turnaround, p[i].completion);
}

printf("Average WT = %.2f\n", totalWT/n);
printf("Average TAT = %.2f\n", totalTAT/n);

fprintf(fout, "Average WT = %.2f\n", totalWT/n);

```

```

    fprintf(fout, "Average TAT = %.2f\n\n", totalTAT/n);
}

int main() {
    FILE *fin, *fout;
    fin = fopen("input.txt", "r");
    fout = fopen("output.txt", "w");

    if(fin == NULL) {
        printf("File input.txt tidak ditemukan!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);
    struct Process p[n];

    for(int i=0; i<n; i++) {
        fscanf(fin, "%s %d %d", p[i].name, &p[i].arrival, &p[i].burst);
    }

    printf("Data berhasil dibaca dari file input.txt\n");
    fprintf(fout, "=== HASIL CPU SCHEDULING ===\n");

    fcfs(p, n, fout);

    fclose(fin);
    fclose(fout);

    printf("\nHasil juga disimpan ke file output.txt\n");
    return 0;
}

```

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma Test Case

1: Proses dengan burst time bervariasi

Jumlah proses: 4

Time quantum: 3

- P1: AT=0, BT=24

- P2: AT=1, BT=3

- P3: AT=2, BT=3

- P4: AT=3, BT=6

Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

- P1: AT=0, BT=10

- P2: AT=0, BT=5

- P3: AT=0, BT=8

- P4: AT=0, BT=12

- P5: AT=0, BT=6

Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8

- P2: AT=2, BT=4

- P3: AT=4, BT=9

- P4: AT=6, BT=5

- P5: AT=8, BT=3

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano cpu_scheduling_experiment.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc cpu_scheduling_experiment.c -o cpu_scheduling_experiment
senjaramadhani@LAPTOP-823QCJCF:~$ ./cpu_scheduling_experiment

===== TEST CASE 1: Burst Time Bervariasi =====

=== FCFS ===
Proses AT      BT      WT      TAT      CT
P1      0      24      0      24      24
P2      1      3      23      26      27
P3      2      3      25      28      30
P4      3      6      27      33      36
Average WT = 18.75 | Average TAT = 27.75

=== SJF (Non-preemptive) ===
Proses AT      BT      WT      TAT      CT
P1      0      24      0      24      24
P2      1      3      23      26      27
P3      2      3      25      28      30
P4      3      6      27      33      36
Average WT = 18.75 | Average TAT = 27.75

=== Round Robin (q=3) ===
Proses AT      BT      WT      TAT      CT
P1      0      24      12      36      36
P2      1      3      2      5      6
P3      2      3      4      7      9
P4      3      6      9      15      18
Average WT = 6.75 | Average TAT = 15.75

===== TEST CASE 2: Proses Datang Bersamaan =====

=== FCFS ===
Proses AT      BT      WT      TAT      CT
P1      0      10      0      10      10
P2      0      5      10      15      15
P3      0      8      15      23      23
P4      0      12      23      35      35
```

```

=== SJF (Non-preemptive) ===
Proses AT BT WT TAT CT
P1 0 10 19 29 29
P2 0 5 0 5 5
P3 0 8 11 19 19
P4 0 12 29 41 41
P5 0 6 5 11 11
Average WT = 12.80 | Average TAT = 21.80

=== Round Robin (q=4) ===
Proses AT BT WT TAT CT
P1 0 10 27 37 37
P2 0 5 20 25 25
P3 0 8 21 29 29
P4 0 12 29 41 41
P5 0 6 29 35 35
Average WT = 25.20 | Average TAT = 33.40

===== TEST CASE 3: Proses Datang Berinterval =====

=== FCFS ===
Proses AT BT WT TAT CT
P1 0 8 0 8 8
P2 2 4 6 10 12
P3 4 9 8 17 21
P4 6 5 15 20 26
P5 8 3 18 21 29
Average WT = 9.40 | Average TAT = 15.20

=== SJF (Non-preemptive) ===
Proses AT BT WT TAT CT
P1 0 8 0 8 8
P2 2 4 9 13 15
P3 4 9 16 25 29
P4 6 5 9 14 20
P5 8 3 0 3 11
Average WT = 6.80 | Average TAT = 12.60

=== Round Robin (q=2) ===
Proses AT BT WT TAT CT
P1 0 8 18 26 26
P2 2 4 8 12 14
P3 4 9 16 25 29
P4 6 5 13 18 24
P5 8 3 8 11 19
Average WT = 12.60 | Average TAT = 18.40
senjaramadhani@LAPTOP-823QCJCF:~$

```

Source code:

```
#include <stdio.h>
```

```

struct Process {
    char name[5];
    int arrival, burst, waiting, turnaround, completion;
};

```

```
// ===== FCFS =====
```

```

void fcfs(struct Process p[], int n) {
    int time = 0;
    float totalWT = 0, totalTAT = 0;

```

```

printf("\n=== FCFS ===\n");
printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");

for(int i=0;i<n;i++){
    if(time < p[i].arrival)
        time = p[i].arrival;
    p[i].waiting = time - p[i].arrival;
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = p[i].completion - p[i].arrival;

    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;

    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].name, p[i].arrival, p[i].burst, p[i].waiting, p[i].turnaround,
p[i].completion);
}

printf("Average WT = %.2f | Average TAT = %.2f\n", totalWT/n, totalTAT/n);
}

// ===== SJF Non-Preemptive =====
void sjf(struct Process p[], int n) {
    int completed = 0, time = 0, min_bt, pos;
    int isCompleted[n];
    for(int i=0;i<n;i++) isCompleted[i]=0;

```

```
float totalWT=0, totalTAT=0;
```

```
printf("\n=== SJF (Non-preemptive) ===\n");
```

```
printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");
```

```
while(completed != n){
```

```
    min_bt = 9999; pos = -1;
```

```
    for(int i=0;i<n;i++){
```

```
        if(p[i].arrival <= time && isCompleted[i]==0 && p[i].burst < min_bt){
```

```
            min_bt = p[i].burst;
```

```
            pos = i;
```

```
        }
```

```
    }
```

```
    if(pos == -1){
```

```
        time++;
```

```
        continue;
```

```
    }
```

```
    p[pos].waiting = time - p[pos].arrival;
```

```
    time += p[pos].burst;
```

```
    p[pos].completion = time;
```

```
    p[pos].turnaround = p[pos].completion - p[pos].arrival;
```

```
    isCompleted[pos] = 1;
```

```
    completed++;
```

```
}
```

```
for(int i=0;i<n;i++){
```

```
    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
```

```

        p[i].name, p[i].arrival, p[i].burst, p[i].waiting, p[i].turnaround,
p[i].completion);
        totalWT += p[i].waiting;
        totalTAT += p[i].turnaround;
    }

    printf("Average WT = %.2f | Average TAT = %.2f\n", totalWT/n, totalTAT/n);
}

```

```

// ===== Round Robin =====

```

```

void rr(struct Process p[], int n, int quantum) {
    int remaining[n];
    int time = 0, done, completed = 0;
    float totalWT=0, totalTAT=0;
    for(int i=0;i<n;i++) remaining[i]=p[i].burst;

    printf("\n=== Round Robin (q=%d) ===\n", quantum);
    printf("Proses\tAT\tBT\tWT\tTAT\tCT\n");

    while(completed < n){
        done = 1;
        for(int i=0;i<n;i++){
            if(remaining[i] > 0 && p[i].arrival <= time){
                done = 0;
                if(remaining[i] > quantum){
                    time += quantum;
                    remaining[i] -= quantum;
                }
            }
        }
        if(done){
            completed++;
            totalWT += p[completed-1].waiting;
            totalTAT += p[completed-1].turnaround;
        }
    }
}

```

```

        } else {
            time += remaining[i];
            p[i].completion = time;
            p[i].turnaround = p[i].completion - p[i].arrival;
            p[i].waiting = p[i].turnaround - p[i].burst;
            remaining[i] = 0;
            completed++;
        }
    }
}

if(done) time++;
}

for(int i=0;i<n;i++){
    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].name, p[i].arrival, p[i].burst, p[i].waiting, p[i].turnaround,
p[i].completion);
    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;
}

printf("Average WT = %.2f | Average TAT = %.2f\n", totalWT/n, totalTAT/n);
}

// ===== MAIN =====

int main() {
    printf("\n===== TEST CASE 1: Burst Time Bervariasi =====\n");
    struct Process tc1[] = {{"P1",0,24}, {"P2",1,3}, {"P3",2,3}, {"P4",3,6}};

```

```

int n1 = 4;
fcfs(tc1, n1);
struct Process tc1_sjf[] = {{"P1",0,24}, {"P2",1,3}, {"P3",2,3}, {"P4",3,6}};
sjf(tc1_sjf, n1);
struct Process tc1_rr[] = {{"P1",0,24}, {"P2",1,3}, {"P3",2,3}, {"P4",3,6}};
rr(tc1_rr, n1, 3);

printf("\n\n===== TEST CASE 2: Proses Datang Bersamaan =====\n");
struct Process tc2[] = {{"P1",0,10}, {"P2",0,5}, {"P3",0,8}, {"P4",0,12},
{"P5",0,6}};
int n2 = 5;
fcfs(tc2, n2);
struct Process tc2_sjf[] = {{"P1",0,10}, {"P2",0,5}, {"P3",0,8}, {"P4",0,12},
{"P5",0,6}};
sjf(tc2_sjf, n2);
struct Process tc2_rr[] = {{"P1",0,10}, {"P2",0,5}, {"P3",0,8}, {"P4",0,12},
{"P5",0,6}};
rr(tc2_rr, n2, 4);

printf("\n\n===== TEST CASE 3: Proses Datang Berinterval =====\n");
struct Process tc3[] = {{"P1",0,8}, {"P2",2,4}, {"P3",4,9}, {"P4",6,5},
{"P5",8,3}};
int n3 = 5;
fcfs(tc3, n3);
struct Process tc3_sjf[] = {{"P1",0,8}, {"P2",2,4}, {"P3",4,9}, {"P4",6,5},
{"P5",8,3}};
sjf(tc3_sjf, n3);

```

```

struct Process tc3_rr[] = {"P1",0,8}, {"P2",2,4}, {"P3",4,9}, {"P4",6,5},
{"P5",8,3}};

rr(tc3_rr, n3, 2);

return 0;
}

```

Tugas 5: Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa

a. Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS (First Come First Served) bekerja paling baik ketika semua proses memiliki waktu burst yang hampir sama dan datang secara berurutan tanpa perbedaan arrival time yang ekstrem.

b. Mengapa SJF hampir selalu memberikan Average WT terkecil?

Karena SJF (Shortest Job First) selalu mengeksekusi proses dengan burst time paling kecil terlebih dahulu, sehingga proses singkat tidak perlu menunggu proses panjang selesai.

c. Apa trade-off dari Round Robin?

Round Robin memberikan pembagian waktu yang adil (fairness) antar proses, tetapi:

- Terjadi banyak context switch, yang meningkatkan overhead CPU.
- Jika quantum terlalu kecil, maka efisiensi menurun.
- Jika quantum terlalu besar, maka perilakunya menjadi mirip FCFS (tidak lagi adil).

2. Context Switching

- a. Algoritma mana yang paling banyak melakukan context switch?

Round Robin memiliki jumlah context switch paling banyak karena CPU terus berpindah antar proses berdasarkan quantum waktu.

- b. Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi? Semakin kecil quantum, semakin banyak context switch dan lebih responsif, tapi dengan biaya efisiensi CPU. Semakin besar quantum, semakin sedikit context switch, tapi respons time menurun.

3. Fairness

- a. Algoritma mana yang paling "adil" untuk semua proses? • Proses mana yang paling dirugikan di algoritma FCFS?

Round Robin paling adil karena setiap proses mendapat jatah CPU dengan durasi quantum yang sama.

- b. Apakah ada kemungkinan starvation di SJF?

Proses dengan burst time pendek yang datang terlambat.

Karena proses panjang yang datang lebih dulu akan memonopoli CPU, menyebabkan proses pendek menunggu lama (*convoy effect*).

4. Real World Application

- a. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Algoritma yang cocok: Round Robin (RR)

- b. Untuk render video (task besar), algoritma mana yang cocok?

Algoritma yang cocok: FCFS atau SJF (Shortest Job First)

- c. Untuk OS desktop (interactive), algoritma mana yang cocok?

Algoritma yang cocok: Round Robin (dengan prioritas dinamis)

